Programming project 2: Bayesian networks with LLMs

Due: see Canvas
Group work: You may work in groups of 1-3. Include all group members when submitting to Gradescope.

The goal of this project is to re-implement the same Python class as in Project 1 to solve the disease diagnosis problem but by prompting an LLM (instead of calling the Bayesian network functions). The diagnosis is to be made using the same Bayesian network from the first project.

The class `Diagnostics` should still have a constructor and a single method:

```
diagnose (visit_to_asia, smoking, xray_result, dyspnea)
```

where the parameters are strings that describe the symptoms. The method should return a list of two items:

1. The name of the disease that is most likely given the symptoms (a string): “TB”, “Cancer”, or “Bronchitis”. You must use exactly these words.
2. The probability of the most likely disease (a number between 0 and 1).

Refer to Project 1’s requirements for the details of the class `Diagnostics.` The Diagnostics class will be called by the same GUI. You can use the same completed GUI code and a skeleton of the diagnostics class from the first project on Canvas (`diagnostics_gui.py` and `diagnostics.py` on Canvas).

Choice of LLMs

You are allowed to call any LLM. Each LLM provider (Google/OpenAI/Hugging Face) requires you to use their own API. Some choices:

1. **NRP-managed LLMs (recommended)**. CSUF students have free access to some models hosted on the National Research Platform (NRP). OpenAI's gpt-oss-120b model can solve this problem (model = "gpt-oss"). These models have to be prompted using the OpenAI API but as they are hosted on NRP, you will need to use an API key from NRP (i.e., not ones issued by OpenAI). You should have already created such a key during the class work. If you are unable to generate an API key, you can use the one posted by the instructor on Canvas.
2. **Google Gemini**. The Gemini 2.5 Flash can solve this problem with the appropriate prompt. You will have to apply for a Google Gemini API key. As this problem only has text and each call to method diagnose can be solved using a single LLM call, the free tier **may be** sufficient for this project. You can test your prompts on Google’s AI Studio and include only the final working version in your code. You may have to use your

personal gmail account to generate the key, not the @csu.fullerton.edu email as that tends to give resource exceeded errors.

3. **OpenAI models.** The lower-end gpt-5-nano model can solve this problem. You will need an API key from OpenAI. OpenAI does not have an official free tier. However, it appears that you can still prompt their model a certain number of times even without any credits (i.e., you don't need to pay for a subscription just for this project). Note that you can test your prompts on TitanGPT for free as many times you like during development, and then only include the final working version of the prompt in your code.
4. **A local model.** If you have a computer with a GPU and over 16GB of RAM, you can download, install, and prompt an open-source model on your own computer. This does not require getting a third party API key. For instance, the Mistral-7B-v0.3 model available from Hugging Face can solve this problem (and can be queried using the Transformers API as shown in class). Running a model locally will require installing PyTorch as described here.

Prompts:

Note that the prompt has to include all the information that the LLM will need to solve the problem. Thus, the prompt has to include both the information in the Bayesian network and the evidence. You can do an internet search for *probabilistic reasoning in LLMs* to see some ways on how to represent a Bayesian network in a prompt.

Structured output:

The diagnose method should return only two values (a specific string, and a probability). You will have to get this from the LLM response. There are three ways of doing this:

1. Parse the text output of the LLM with your own code to extract the name of the disease and probability. This is cumbersome!
2. Include in the prompt instructions to generate only the required information and not to produce any other text.
3. Specify an output template for structured output. This is the most reliable way. See slides on prompting on Canvas for examples.

Submit via Gradescope:

Submission is via Gradescope. Upload the following THREE files:

1. `diagnostics.py`. The name of the class must be `Diagnostics`. These are case-sensitive.
    a. Include comments and print statements in your code for the intermediate steps.
    b. **Do NOT include your private API key**. Delete this from your code before submission, or better yet, store it in your own desktop environment outside the code.
2. A screenshot of the output produced by your code as it is running (JPG or PNG file)
3. A short report in PDF format answering the following questions:

a. Why did you choose the particular LLM? (1-2 sentences)
b. The major challenge in completing your project (1-4 sentences)
c. Something surprising you learned about LLMs from this project (1-3 sentences)

Gradescope will run a few unit tests automatically and I will complete the grading manually. Gradescope requires that all 3 files be submitted at the same time. Include all group members when submitting to Gradescope.